

Werkzeuge

SQLite

Inhaltsverzeichnis

SQLite - Werkzeughandbuch	4
Inhalte	4
Was ist SQLite?	5
Eigenschaften	5
Typische Einsatzbereiche	5
Abgrenzung	5
SQLite installieren	6
macOS	6
Linux	6
Windows	6
Prüfung	6
SQLite CLI	7
Neue Datenbank öffnen oder anlegen	7
Hilfe	7
Tabellen anzeigen	7
Schema anzeigen	7
Ausgabe übersichtlicher darstellen	7
SQLite verlassen	7
Merke	7
Datenbank anlegen	8
CLI	8
Speichern	8
Dateiendungen	8
DB Browser for SQLite	9
Typischer Ablauf	9
Wichtige Bereiche	9
Unterrichtshinweis	9
SQL-Dateien ausführen	10
CLI	10
Vorteil	10
Empfehlung	10
Fremdschlüssel in SQLite	11
Aktivieren	11
Status prüfen	11
Beispiel	11
Wichtig	11
Datentypen und Type Affinity	12
Beispiel	12
Wichtig	12
STRICT Tables	12
INTEGER PRIMARY KEY	13
Automatische ID	13
Empfehlung	13
AUTOINCREMENT	14
Unterschied	14

Konsequenz	14
Empfehlung	14
TRUNCATE TABLE in SQLite	15
SQLite-Variante	15
Achtung	15
ALTER TABLE - Besonderheiten	16
Tabelle umbenennen	16
Spalte umbenennen	16
Spalte hinzufügen	16
Einschränkung	16
CSV importieren	17
DB Browser for SQLite	17
SQLite CLI	17
Wichtig	17
CSV exportieren	18
SQLite CLI	18
DB Browser for SQLite	18
Einsatz	18
Backup	19
SQLite CLI	19
Wiederherstellung	19
Empfehlung	19
Merke	19
Transaktionen in SQLite	20
Bedeutung	20
Indizes in SQLite	21
Abfrageplan	21
Views in SQLite	22
Einordnung	22
Trigger in SQLite	23
Einsatzgebiete	23
Hinweis	23
Stored Procedures und SQLite	24
Konsequenz	24
Typische Fehler	25
no such table	25
no such column	25
UNIQUE constraint failed	25
FOREIGN KEY constraint failed	25
database is locked	25
syntax error	25
Vorgehen	26
HowTo - SQLite im Unterricht	27
Neue Übungsdatenbank	27
Fremdschlüssel aktivieren	27

Schema ausführen	27
Tabellen prüfen	27
Daten anzeigen	27
CRUD üben	27
Beenden	27
Empfohlener Lernweg	27
SQLite Cheatsheet	28
Start	28
Hilfe	28
Tabellen	28
Schema	28
SQL-Datei	28
Format	28
Fremdschlüssel	28
Backup	28
Import CSV	28
Beenden	28
Übungen	29
1 - Datenbank öffnen	29
2 - Fremdschlüssel	29
3 - Schema	29
4 - CRUD	29
5 - CSV	29
6 - Backup	29
7 - Index	29
Quellen	30

SQLite - Werkzeughandbuch

Dieser Bereich beschreibt den praktischen Einsatz von SQLite.

Die allgemeinen SQL-Grundlagen befinden sich unter:

[Grundlagen/SQL/](#)

Hier geht es ausschließlich um SQLite-spezifische Besonderheiten und Werkzeuge.

Inhalte

- SQLite installieren
- SQLite CLI verwenden
- DB Browser for SQLite verwenden
- Datenbanken anlegen und öffnen
- SQL-Dateien ausführen
- CSV importieren und exportieren
- Backups erstellen
- Fremdschlüssel aktivieren
- Datentypen und Type Affinity
- INTEGER PRIMARY KEY
- AUTOINCREMENT
- Besonderheiten bei ALTER TABLE
- Ersatz für TRUNCATE TABLE
- typische Fehler
- praktische Übungen

Was ist SQLite?

SQLite ist ein relationales Datenbanksystem, das ohne separaten Datenbankserver arbeitet.

Eine SQLite-Datenbank besteht typischerweise aus einer einzelnen Datei.

Beispiel:

schulverwaltung.db

Eigenschaften

SQLite ist:

- leichtgewichtig,
- serverlos,
- dateibasiert,
- weit verbreitet,
- gut für lokale Anwendungen und Unterricht geeignet.

Typische Einsatzbereiche

- Desktop-Programme
- mobile Apps
- lokale Werkzeuge
- Prototypen
- Tests
- Unterricht

Abgrenzung

SQLite ist ein konkretes Datenbanksystem.

SQL ist die Sprache, mit der relationale Datenbanken beschrieben und abgefragt werden.

SQL ist der Standard bzw. die Sprache. SQLite ist eine konkrete Implementierung.

SQLite installieren

macOS

SQLite ist auf vielen macOS-Systemen bereits vorhanden.

Prüfen:

```
sqlite3 --version
```

Falls SQLite nicht vorhanden ist oder eine aktuelle Version benötigt wird:

```
brew install sqlite
```

Linux

Beispiel Debian/Ubuntu:

```
sudo apt update
```

```
sudo apt install sqlite3
```

Windows

SQLite kann als Kommandozeilenwerkzeug installiert werden.

Alternativ kann DB Browser for SQLite verwendet werden.

Prüfung

```
sqlite3 --version
```

Wenn eine Versionsnummer erscheint, ist das Werkzeug verfügbar.

SQLite CLI

Die SQLite-Kommandozeile wird mit `sqlite3` gestartet.

Neue Datenbank öffnen oder anlegen

```
sqlite3 schulverwaltung.db
```

Existiert die Datei noch nicht, wird sie beim ersten Speichern angelegt.

Hilfe

```
.help
```

Tabellen anzeigen

```
.tables
```

Schema anzeigen

```
.schema
```

Ausgabe übersichtlicher darstellen

```
.headers on  
.mode column
```

SQLite verlassen

```
.quit
```

Merke

Befehle, die mit einem Punkt beginnen, sind Befehle der SQLite-CLI und **kein SQL**.

Beispiel:

```
.tables
```

ist SQLite-CLI.

```
SELECT * FROM Schueler;
```

ist SQL.

Datenbank anlegen

CLI

sqlite3 schule.db

Danach kann direkt SQL eingegeben werden.

```
CREATE TABLE Klasse (  
    KlasseID INTEGER PRIMARY KEY,  
    Bezeichnung TEXT NOT NULL UNIQUE  
);
```

Speichern

SQLite schreibt Änderungen grundsätzlich in die Datenbankdatei.

Die Datei kann danach mit anderen SQLite-Werkzeugen geöffnet werden.

Dateiendungen

Übliche Endungen sind:

- .db
- .sqlite
- .sqlite3

Die Endung ändert nicht das Datenbankformat.

DB Browser for SQLite

DB Browser for SQLite ist eine grafische Oberfläche zur Arbeit mit SQLite-Datenbanken.

Typischer Ablauf

1. Programm starten.
2. **Neue Datenbank** wählen.
3. Speicherort und Dateinamen festlegen.
4. Tabellen anlegen oder SQL ausführen.
5. Daten kontrollieren.
6. Änderungen speichern.

Wichtige Bereiche

Datenbankstruktur

Zeigt:

- Tabellen
- Spalten
- Indizes
- Trigger
- Views

Daten durchsuchen

Zeigt Datensätze tabellarisch.

SQL ausführen

Hier können SQL-Anweisungen geschrieben und ausgeführt werden.

Unterrichtshinweis

Für den Einstieg ist der SQL-Editor besonders wichtig.

Die Schülerinnen und Schüler sollen SQL-Befehle bewusst ausführen und nicht ausschließlich Tabellen über grafische Dialoge erzeugen.

SQL-Dateien ausführen

SQL-Befehle können in Dateien gespeichert werden.

Beispiel:

```
schema.sql
```

CLI

```
sqlite3 schule.db < schema.sql
```

Oder innerhalb der SQLite-CLI:

```
.read schema.sql
```

Vorteil

Damit können Datenbanken reproduzierbar aufgebaut werden.

Ein SQL-Skript kann enthalten:

- CREATE TABLE
- CREATE INDEX
- INSERT
- Beispieldaten

Empfehlung

Datenbankschema und Beispieldaten nach Möglichkeit als SQL-Dateien versionieren.

Fremdschlüssel in SQLite

SQLite unterstützt Fremdschlüssel.

Sie müssen jedoch in vielen Nutzungssituationen explizit aktiviert werden.

Aktivieren

```
PRAGMA foreign_keys = ON;
```

Status prüfen

```
PRAGMA foreign_keys;
```

Ergebnis:

1

bedeutet aktiviert.

Beispiel

```
CREATE TABLE Klasse (  
    KlasseID INTEGER PRIMARY KEY,  
    Bezeichnung TEXT NOT NULL  
);  
  
CREATE TABLE Schueler (  
    SchuelerID INTEGER PRIMARY KEY,  
    Name TEXT NOT NULL,  
    KlasseID INTEGER,  
    FOREIGN KEY (KlasseID)  
        REFERENCES Klasse(KlasseID)  
);
```

Wichtig

Ein definierter Fremdschlüssel schützt nur dann zuverlässig, wenn die Fremdschlüsselprüfung aktiviert ist.

Datentypen und Type Affinity

SQLite behandelt Datentypen flexibler als viele andere relationale Datenbanksysteme.

SQLite verwendet sogenannte **Type Affinities**.

Wichtige Gruppen sind:

- INTEGER
- REAL
- TEXT
- BLOB
- NUMERIC

Beispiel

```
CREATE TABLE Messwert (  
    MesswertID INTEGER PRIMARY KEY,  
    Wert REAL,  
    Einheit TEXT  
);
```

Wichtig

SQLite erzwingt Datentypen in klassischen Tabellen weniger streng als viele andere Datenbanksysteme.

Für Unterricht und Datenqualität gilt trotzdem:

Spalten sollten fachlich passende Datentypen erhalten.

STRICT Tables

Neuere SQLite-Versionen unterstützen STRICT-Tabellen.

```
CREATE TABLE Beispiel (  
    ID INTEGER PRIMARY KEY,  
    Name TEXT NOT NULL  
) STRICT;
```

Damit werden Datentypen strenger kontrolliert.

INTEGER PRIMARY KEY

INTEGER PRIMARY KEY besitzt in SQLite eine besondere Bedeutung.

```
CREATE TABLE Schueler (  
    SchuelerID INTEGER PRIMARY KEY,  
    Name TEXT NOT NULL  
);
```

Die Spalte wird dabei mit der internen rowid verbunden.

Automatische ID

Wird keine ID angegeben, kann SQLite eine passende Ganzzahl erzeugen.

```
INSERT INTO Schueler (Name)  
VALUES ('Mia');
```

Empfehlung

Für einfache lokale Datenbanken ist

INTEGER PRIMARY KEY

meist ausreichend.

AUTOINCREMENT ist nicht automatisch notwendig.

AUTOINCREMENT

SQLite unterstützt:

INTEGER PRIMARY KEY AUTOINCREMENT

Beispiel:

```
CREATE TABLE Schueler (  
    SchuelerID INTEGER PRIMARY KEY AUTOINCREMENT,  
    Name TEXT NOT NULL  
);
```

Unterschied

INTEGER PRIMARY KEY kann bereits automatisch neue IDs erzeugen.

AUTOINCREMENT verändert die Regel, nach der neue IDs vergeben werden, und verhindert insbesondere die Wiederverwendung bestimmter bereits vergebener Werte.

Konsequenz

AUTOINCREMENT verursacht zusätzlichen Verwaltungsaufwand.

Empfehlung

AUTOINCREMENT nur verwenden, wenn die strengere ID-Vergabe tatsächlich benötigt wird.

TRUNCATE TABLE in SQLite

SQLite unterstützt TRUNCATE TABLE nicht.

Folgendes ist deshalb ungültig:

```
TRUNCATE TABLE Schueler;
```

SQLite-Variante

Alle Datensätze löschen:

```
DELETE FROM Schueler;
```

Die Tabelle bleibt bestehen.

Achtung

DELETE FROM und TRUNCATE TABLE sind konzeptionell nicht identisch.

Das SQL-Grundlagenkapitel behandelt die allgemeine Bedeutung von TRUNCATE TABLE.

Dieses Kapitel beschreibt nur die SQLite-spezifische Umsetzung.

ALTER TABLE - Besonderheiten

SQLite unterstützt wichtige ALTER TABLE-Operationen.

Beispiele:

Tabelle umbenennen

```
ALTER TABLE Schueler  
RENAME TO Lernende;
```

Spalte umbenennen

```
ALTER TABLE Schueler  
RENAME COLUMN Name TO Nachname;
```

Spalte hinzufügen

```
ALTER TABLE Schueler  
ADD COLUMN Email TEXT;
```

Einschränkung

Komplexe Schemaänderungen können je nach SQLite-Version und gewünschter Änderung eine neue Tabelle erfordern.

Typischer Ablauf:

1. neue Tabelle anlegen,
2. Daten übertragen,
3. alte Tabelle löschen,
4. neue Tabelle umbenennen.

Vor solchen Änderungen immer ein Backup erstellen.

CSV importieren

DB Browser for SQLite

Typischer Ablauf:

1. Datenbank öffnen.
2. Importfunktion wählen.
3. CSV-Datei auswählen.
4. Trennzeichen prüfen.
5. Spaltennamen prüfen.
6. Datentypen prüfen.
7. Import durchführen.
8. Ergebnis kontrollieren.

SQLite CLI

```
.mode csv  
.import daten.csv Schueler
```

Wichtig

Vor dem Import prüfen:

- Zeichencodierung
- Trennzeichen
- Kopfzeile
- Datentypen
- leere Werte
- eindeutige Schlüssel

Ein erfolgreicher Import bedeutet nicht automatisch, dass die Daten fachlich korrekt sind.

CSV exportieren

SQLite CLI

```
.headers on  
.mode csv  
.output schueler.csv  
SELECT * FROM Schueler;  
.output stdout
```

DB Browser for SQLite

Abfragen oder Tabellen können über die Exportfunktionen als CSV gespeichert werden.

Einsatz

CSV eignet sich besonders für:

- Tabellenkalkulation,
- Datenaustausch,
- kleinere Analysen,
- Sicherung einzelner Abfrageergebnisse.

Backup

Eine SQLite-Datenbank ist häufig nur eine Datei.

Trotzdem sollte sie nicht beliebig während laufender Schreibzugriffe kopiert werden.

SQLite CLI

```
.backup sicherung.db
```

Wiederherstellung

Eine Sicherungsdatei kann anschließend wie jede andere SQLite-Datenbank geöffnet werden.

Empfehlung

Vor größeren Schemaänderungen:

1. Datenbank schließen oder konsistent sichern.
2. Backup erzeugen.
3. Änderung durchführen.
4. Ergebnis prüfen.

Merke

Eine Änderung ohne Backup ist nur so lange harmlos, bis sie schiefgeht.

Transaktionen in SQLite

SQLite unterstützt Transaktionen.

BEGIN;

UPDATE Konto
SET Guthaben = Guthaben - 10
WHERE KontoID = 1;

UPDATE Konto
SET Guthaben = Guthaben + 10
WHERE KontoID = 2;

COMMIT;

Bei einem Fehler:

ROLLBACK;

Bedeutung

Mehrere Änderungen können damit als gemeinsame Einheit behandelt werden.

Transaktionen gehören fachlich zu den SQL-/Datenbankgrundlagen, werden hier aber zusätzlich praktisch für SQLite gezeigt.

Indizes in SQLite

Index anlegen:

```
CREATE INDEX idx_schueler_name  
ON Schueler(Name);
```

Indizes anzeigen:

```
SELECT name  
FROM sqlite_master  
WHERE type = 'index';
```

Index löschen:

```
DROP INDEX idx_schueler_name;
```

Abfrageplan

SQLite kann den geplanten Zugriff anzeigen:

```
EXPLAIN QUERY PLAN  
SELECT *  
FROM Schueler  
WHERE Name = 'Mia';
```

Das hilft zu erkennen, ob ein Index verwendet wird.

Views in SQLite

SQLite unterstützt Views.

```
CREATE VIEW SchuelerMitKlasse AS  
SELECT  
    Schueler.SchuelerID,  
    Schueler.Name,  
    Klasse.Bezeichnung  
FROM Schueler  
JOIN Klasse  
    ON Schueler.KlasseID = Klasse.KlasseID;
```

Abfrage:

```
SELECT *  
FROM SchuelerMitKlasse;
```

Löschen:

```
DROP VIEW SchuelerMitKlasse;
```

Einordnung

Views sind ein fortgeschrittenes Thema.

Die allgemeinen Konzepte gehören in den SQL-Grundlagenbereich, sobald sie dort eingeführt werden.

Trigger in SQLite

SQLite unterstützt Trigger.

Ein Trigger führt automatisch SQL aus, wenn ein bestimmtes Ereignis eintritt.

Beispiel:

```
CREATE TRIGGER Beispiel
AFTER INSERT ON Schueler
BEGIN
    -- weitere SQL-Anweisungen
END;
```

Einsatzgebiete

- Protokollierung
- automatische Folgeänderungen
- zusätzliche Prüfungen

Hinweis

Trigger sind ein fortgeschrittenes Thema und sollten erst eingesetzt werden, wenn normale SQL-Operationen sicher beherrscht werden.

Stored Procedures und SQLite

SQLite besitzt keine Stored Procedures wie beispielsweise Oracle, PostgreSQL oder SQL Server.

Geschäftslogik wird bei SQLite typischerweise:

- in der Anwendung,
- über Trigger,
- oder über Erweiterungen

umgesetzt.

Konsequenz

Stored Procedures gehören nicht zum SQLite-Unterricht.

Das allgemeine Konzept kann später unter Grundlagen/SQL behandelt werden.

Typische Fehler

no such table

no such table: Schueler

Mögliche Ursache:

- Tabelle nicht angelegt,
 - falsche Datenbank geöffnet,
 - Schreibfehler.
-

no such column

no such column: Klasse

Spaltennamen prüfen.

UNIQUE constraint failed

Ein eindeutiger Wert existiert bereits.

Beispiel:

UNIQUE constraint failed: Benutzer.Email

FOREIGN KEY constraint failed

Ein Fremdschlüssel verweist auf einen ungültigen Datensatz oder eine Löschoperation verletzt eine Beziehung.

database is locked

Eine andere Verbindung verwendet die Datenbank gerade für einen Schreibzugriff.

syntax error

SQL-Syntax prüfen:

- Komma,
- Klammern,
- Schlüsselwörter,
- Anführungszeichen,
- Semikolon.

Vorgehen

1. Fehlermeldung vollständig lesen.
2. kleinste betroffene SQL-Anweisung isolieren.
3. Tabellen- und Spaltennamen prüfen.
4. Schema kontrollieren.
5. Änderung einzeln testen.

HowTo - SQLite im Unterricht

Neue Übungsdatenbank

sqlite3 schule.db

Fremdschlüssel aktivieren

PRAGMA foreign_keys = **ON**;

Schema ausführen

.read Beispiele/schulverwaltung_sqlite.sql

Tabellen prüfen

.tables

Daten anzeigen

SELECT *
FROM Schueler;

CRUD üben

INSERT ...
SELECT ...
UPDATE ...
DELETE ...

Beenden

.quit

Empfohlener Lernweg

1. Tabelle ansehen.
2. SELECT ausführen.
3. INSERT ausführen.
4. UPDATE ausführen.
5. DELETE ausführen.
6. neue Tabelle anlegen.
7. Fremdschlüssel ergänzen.
8. Join ausführen.

SQLite Cheatsheet

Start

```
sqlite3 datenbank.db
```

Hilfe

```
.help
```

Tabellen

```
.tables
```

Schema

```
.schema
```

SQL-Datei

```
.read datei.sql
```

Format

```
.headers on  
.mode column
```

Fremdschlüssel

```
PRAGMA foreign_keys = ON;
```

Backup

```
.backup sicherung.db
```

Import CSV

```
.mode csv  
.import daten.csv Tabelle
```

Beenden

```
.quit
```

Übungen

1 - Datenbank öffnen

Lege eine neue Datenbank `schule.db` an.

2 - Fremdschlüssel

Aktiviere Fremdschlüssel und prüfe den Status.

3 - Schema

Führe die Datei `Beispiele/schulverwaltung_sqlite.sql` aus.

Prüfe anschließend die Tabellen.

4 - CRUD

Füge einen Schüler ein, lies ihn aus, ändere den Namen und lösche den Datensatz wieder.

5 - CSV

Exportiere die Tabelle `Schueler` als CSV.

6 - Backup

Erzeuge eine Sicherungskopie der Datenbank.

7 - Index

Erzeuge einen Index auf `Schueler.Name`.

Vergleiche den Abfrageplan vorher und nachher.

Quellen

Für diesen Werkzeugbereich sollte die offizielle SQLite-Dokumentation als primäre technische Quelle verwendet werden.

Insbesondere relevant sind:

- SQLite Language Reference
- SQLite Datatypes
- Foreign Key Support
- AUTOINCREMENT
- Command Line Shell
- Backup API
- STRICT Tables

Für DB Browser for SQLite sollte die offizielle Projektdokumentation verwendet werden.

Die SQL-Grundkonzepte selbst werden unter Grundlagen/SQL dokumentiert.